Crud

Banco de dados

```sql
1 •   CREATE DATABASE Crud_Java;
2
3 •   USE Crud_Java;
```

📂 Caminho: src/main/resources/application.properties

```
application.properties × | Pessoa.java | PessoaRepository.java | PessoaController.java | index.html

1  spring.application.name=SpringBoot_Projeto04_Crud
2
3  # 1. Conexão com o Banco de Dados
4  spring.datasource.url=jdbc:mysql://localhost:3306/Crud_Java?createDatabaseIfNotExist=true&useSSL=false&serverT
5  spring.datasource.username=root
6  spring.datasource.password=
7
8  # 2. Configurações do Hibernate (JPA)
9  # 'update' faz o Spring criar ou atualizar as tabelas sozinho baseando-se nas suas classes Java
10 spring.jpa.hibernate.ddl-auto=update
11
12 # Mostra as queries SQL no console (ajuda muito a aprender o que o Spring está fazendo por baixo dos panos)
13 spring.jpa.show-sql=true
14
15 # Formata o SQL no console para ficar legível
16 spring.jpa.properties.hibernate.format_sql=true
17
18 # 3. Dialeto do MySQL (opcional nas versões novas, mas bom ter para evitar erros)
19 spring.jpa.database-platform=org.hibernate.dialect.MySQLDialect
```

1. Conexão com o Banco de Dados (O Endereço)

Nesta parte, o Spring identifica para onde ele deve enviar os dados do usuário.

- **Linha 4 (spring.datasource.url)**: O Spring localiza o servidor MySQL no próprio computador (localhost) na porta 3306. Ele procura o banco chamado Crud_Java. Se ele não encontrar esse banco, ele mesmo o cria, graças ao comando createDatabaseIfNotExist=true.

- **Linha 5 e 6 (`username` e `password`)**: O Spring usa essas credenciais para "pedir permissão" ao MySQL. Ele se apresenta como o usuário `root` e entra sem senha, já que o campo está vazio.

---

2. Configurações do Hibernate/JPA (A Construção)

Aqui, o Spring decide como ele vai lidar com as tabelas do banco de dados baseando-se nas suas classes Java (como a classe `Pessoa`).

- **Linha 10 (`ddl-auto=update`)**: O Spring atua como um inspetor. Ele olha para a sua classe Java e olha para o banco de dados. Se faltar uma coluna ou uma tabela no MySQL, ele a cria automaticamente. Se você mudar algo no Java, ele atualiza o banco sem apagar os dados que já estão lá.
- **Linha 13 (`show-sql=true`)**: O Spring se torna "fofoqueiro". Toda vez que ele salvar, editar ou deletar alguém, ele vai escrever no console do IntelliJ o comando SQL exato que ele enviou para o MySQL. Isso ajuda o programador a ver o que está acontecendo "por baixo dos panos".
- **Linha 16 (`format_sql=true`)**: O Spring organiza o texto do SQL no console para que não fique tudo em uma linha só, facilitando a leitura humana.

---

3. Dialeto do MySQL (O Idioma)

Como existem vários tipos de bancos de dados (Oracle, PostgreSQL, MySQL), o Spring precisa saber qual "sotaque" usar.

- **Linha 19 (`database-platform`)**: O Spring ajusta sua comunicação para o "idioma" específico do MySQL. Isso garante que ele não use comandos que o MySQL não entende, evitando erros de sintaxe durante a execução do CRUD.

Resumo do Comportamento do Spring:

1. **Ele lê o arquivo** em `src/main/resources/`.

2. **Ele estabelece a ponte** com o MySQL usando o endereço e o login fornecidos.
3. **Ele prepara o terreno**, criando as tabelas necessárias para que o seu `index.html` tenha onde salvar as pessoas.

📂 Caminho: `src/main/java/com/example/SpringBoot_Projeto04_Crud/model/Pessoa.java`

```
application.properties   Pessoa.java   PessoaRepository.java   PessoaController.java   index.html
1   package com.example.SpringBoot_Projeto04_Crud.model;
2
3   import jakarta.persistence.*;
4
5   @Entity   Maiquel
6   public class Pessoa
7   {
8       @Id     // Chave Primária
9       @GeneratedValue(strategy = GenerationType.IDENTITY)     // O MySQL gera o ID (1, 2, 3...)
10      private Long id;
11
12      private String nome;   2 usages
13
14      private int idade;   2 usages
15
16      // Construtor vazio (obrigatório para o JPA)
17      public Pessoa(){}   1 usage   Maiquel
18
19      // Getters e Setters
20      public Long getId() { return id; }
24
25      public void setId(Long id) { this.id = id; }
29
30      public String getNome() { return nome; }
34
35      public void setNome(String nome) { this.nome = nome; }
39
40      public int getIdade() { return idade; }
44
45      public void setIdade(int idade) { this.idade = idade; }
49  }
50
```

A Perspectiva do Spring: Como ele utiliza a Entidade

O Spring Boot trata a classe como um objeto conectado diretamente ao banco de dados, seguindo estas regras:

1. O Coração da Entidade: O ID

- **`@Id` (Chave Primária):** Este comando avisa ao banco que esta coluna é o "RG" de cada registro. O Spring garante que nenhuma pessoa tenha o mesmo ID que outra. É através desse ID que o sistema sabe quem deve **deletar** ou **editar** quando o usuário clica nos botões da tabela. Sem isso, o Hibernate não aceita a classe como uma entidade.
- **`@GeneratedValue(strategy = GenerationType.IDENTITY)`:** Aqui o Spring instrui o Java a não se preocupar em inventar um número para o ID. O `IDENTITY` passa a responsabilidade para o **MySQL**. Toda vez que uma pessoa nova é salva, o MySQL olha qual foi o último número e gera o próximo automaticamente. Por isso, não é necessário digitar o ID no formulário do HTML.

---

2. O Restante da Estrutura

Para o fluxo de dados funcionar, o Spring exige estes três pilares obrigatórios:

- **Atributos de Classe (`id`, `nome`, `idade`):** São as colunas da tabela no MySQL. O nome definido no Java será o nome da coluna no banco de dados.
- **Construtor Vazio (`public Pessoa() {}`):** É uma exigência do JPA/Hibernate. Quando o Spring busca os dados no MySQL, ele primeiro cria um objeto "vazio" usando esse construtor e só depois preenche os dados. Sem ele, o projeto não inicia.
- **Getters e Setters:** Funcionam como as portas de entrada e saída.
  - O **Setter** é usado quando o usuário digita no formulário HTML e o dado precisa "entrar" no objeto Java.
  - O **Getter** é usado quando o Thymeleaf precisa "ler" o dado do Java para escrevê-lo na tabela HTML.

Resumo do Comportamento do Spring:

1. **Ele reconhece a classe como uma tabela** através da anotação `@Entity`.
2. **Ele gerencia os IDs automaticamente** usando o `@Id` e o `IDENTITY` para evitar registros duplicados no banco.

3. **Ele transporta os dados entre a tela e o Java** usando os Getters e Setters para que o seu formulário e sua tabela funcionem.
4. **Ele usa o construtor vazio como "ponto de nascimento"** do objeto para conseguir transformar os dados que vêm do MySQL em uma pessoa real no Java.

📂 Caminho:

src/main/java/com/example/SpringBoot_Projeto04_Crud/repository/PessoaRepository.java

```
application.properties   Pessoa.java   PessoaRepository.java   PessoaController.java   index.html
1   package com.example.SpringBoot_Projeto04_Crud.repository;
2
3   import com.example.SpringBoot_Projeto04_Crud.model.Pessoa;
4   import org.springframework.data.jpa.repository.JpaRepository;
5   import org.springframework.stereotype.Repository;
6
7   @Repository   2 usages   Maiquel
8   public interface PessoaRepository extends JpaRepository<Pessoa, Long>
9   {
10      // Aqui já temos os métodos save(), findAll(), deleteById(), etc.
11  }
12
13  // Cria a tabela pessoa usando o Molde do Pessoa.Java
```

Como o Spring Cria a Tabela

O Spring Boot utiliza o **PessoaRepository** como o "braço direito" da Entidade. Ele não é uma classe comum, mas sim uma **interface** que conecta o seu código Java diretamente ao motor de banco de dados (Hibernate).

Aqui está a explicação técnica de como ele funciona:

1. A Herança do Poder: extends JpaRepository

"O Spring utiliza esta interface para criar e gerenciar a tabela 'pessoa' no banco de dados, utilizando a classe Pessoa.java como

molde e fornecendo automaticamente todos os métodos de CRUD (Salvar, Ler, Alterar e Excluir)."

2. O Mapeamento: `<Pessoa, Long>`

O Spring lê esses dois parâmetros para entender a estrutura da tabela:

- **Pessoa**: O Spring olha para este parâmetro e entende: "Vou usar a classe `Pessoa.java` como o molde oficial para criar e gerenciar a tabela no MySQL".
- **Long**: O Spring identifica que o tipo de dado da Chave Primária (o ID) definido lá na Entidade é um número do tipo `Long`.

3. A Automatização: `@Repository`

O Spring marca esta interface com a anotação `@Repository` para que ele possa "injetar" essa ferramenta em outros lugares, como no seu Controller. É através daqui que o Spring ganha o poder de traduzir comandos Java em comandos que o MySQL entende.

Resumo do Comportamento do Spring (Repository):

1. **Ele lê o Molde**: O Spring utiliza as informações da classe `Pessoa.java` para construir a tabela fisicamente no banco de dados.
2. **Ele cria as ferramentas**: Ao detectar o `JpaRepository`, o Spring gera automaticamente toda a lógica necessária para conversar com o MySQL.
3. **Ele executa a persistência**: O Spring usa esta interface como uma ponte: quando você chama um `.save()` no Java, o Repository traduz isso para um `INSERT` ou `UPDATE` no banco de dados.

📂 Caminho:

src/main/resources/templates/index.html

properties | Pessoa.java | PessoaRepository.java | PessoaController.java | index.html | style.css

```html
<!DOCTYPE html>
<!-- Define que o documento é HTML5 -->

<html xmlns:th="http://www.thymeleaf.org">
<!-- Ativa o Thymeleaf (th:...) para usar variáveis vindas do Controller -->

<head>
    <meta charset="UTF-8">
    <title>Meu Primeiro CRUD Spring</title>

    <!-- Importa o CSS da pasta static/css -->
    <!-- O @{} é do Thymeleaf para resolver o caminho -->
    <link rel="stylesheet" th:href="@{/css/style.css}">
</head>

<body>

<div class="container">
    <h2>Cadastro de Usuários</h2>

    <!-- FORMULÁRIO -->
    <!-- th:action → para onde os dados vão (mapeia com @PostMapping("/salvar")) -->
    <!-- th:object="${pessoa}" → conecta o formulário com o objeto Pessoa do Controller -->
    <form th:action="@{/pessoas/salvar}" th:object="${pessoa}" method="POST">

        <!-- Campo escondido (ID) -->
        <!-- MUITO IMPORTANTE -->
        <!-- Se tiver ID → UPDATE -->
        <!-- Se NÃO tiver ID → INSERT -->
        <input type="hidden" th:field="*{id}" />
```

```
30          <input type="hidden" th:field="*{id}" />
31
32          <!-- Campo nome -->
33          <!-- th:field="*{nome}" conecta com atributo "nome" da classe Pessoa -->
34          <input type="text" th:field="*{nome}" placeholder="Digite o nome" required />
35
36          <!-- Campo idade -->
37          <input type="number" th:field="*{idade}" placeholder="Idade" required />
38
39          <!-- Botão que envia o formulário -->
40          <button type="submit">Gravar Dados</button>
41      </form>
42
43      <h2>Lista de Cadastrados</h2>
44
45      <!-- TABELA COM OS DADOS -->
46      <table>
47          <thead>
48          <tr>
49              <th>Nome</th>
50              <th>Idade</th>
51              <th>Ações</th>
52          </tr>
53          </thead>
54
55          <tbody>
```

```
55          <tbody>
56
57          <!-- LOOP -->
58          <!-- Percorre a lista "ListaDePessoas" que veio do Controller -->
59          <!-- Cada item vira "p" -->
60          <tr th:each="p : ${ListaDePessoas}">
61
62              <!-- Mostra o nome -->
63              <td th:text="${p.nome}"></td>
64
65              <!-- Mostra a idade -->
66              <td th:text="${p.idade}"></td>
67
68              <td>
69
70                  <!-- BOTÃO EDITAR -->
71                  <!-- Monta uma URL dinâmica: /pessoas/editar/ID -->
72                  <!-- Vai chamar o método @GetMapping("/editar/{id}") -->
73                  <a class="btn-edit"
74                     th:href="@{/pessoas/editar/{id}(id=${p.id})}">
75                      Editar
76                  </a>
```

```
72                <!-- Vai chamar o método @GetMapping("/editar/{id}") -->
73                <a class="btn-edit"
74                   th:href="@{/pessoas/editar/{id}(id=${p.id})}">
75                    Editar
76                </a>
77
78                <!-- BOTÃO DELETAR -->
79                <!-- Também monta URL dinâmica -->
80                <!-- Vai chamar @GetMapping("/deletar/{id}") -->
81                <!-- confirm → pergunta antes de deletar -->
82                <a class="btn-delete"
83                   th:href="@{/pessoas/deletar/{id}(id=${p.id})}"
84                   onclick="return confirm('Deseja realmente excluir?')">
85                    Excluir
86                </a>
87
88            </td>
89        </tr>
90
91        </tbody>
92    </table>
93 </div>
94
95 </body>
96 </html>
```

Explicação Técnica: Thymeleaf e o Fluxo de Dados

O Spring Boot utiliza o Thymeleaf para garantir que a comunicação entre o navegador e o servidor seja exata. Abaixo, detalha-se como ele processa essas funções avançadas:

1. A Função do `th:field="*{nome}"` (Simplificação e Precisão)

O Spring utiliza o `th:field` como um "atalho inteligente" para garantir a **precisão** do mapeamento. Em vez de o desenvolvedor escrever comandos manuais, o Spring executa três tarefas de uma só vez:

- **`id="nome"`**: O Spring gera um ID para o campo automaticamente.
- **`name="nome"`**: Ele define o nome do campo para que o Controller saiba exatamente em qual atributo da classe salvar o dado.

- **value="..."**: Ele realiza o **Data Binding** (vínculo de dados), preenchendo a caixa de texto automaticamente caso o objeto já possua dados (como na edição).

Resumo: O th:field garante que o HTML esteja perfeitamente "plugado" ao atributo da classe Java, evitando erros de digitação e garantindo que o dado chegue na "gaveta" correta.

2. A Chamada do ID Dinâmico: (id=${p.id})

O Spring interpreta o trecho th:href="@{/pessoas/editar/{id}(id=${p.id})}" para criar caminhos específicos:

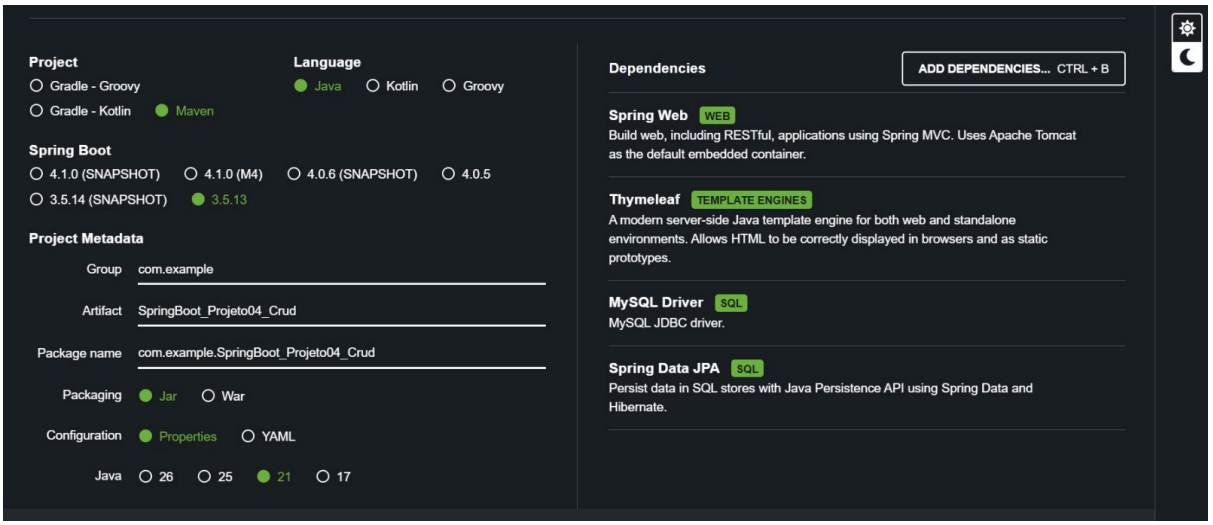
- **A Substituição**: O {id} na URL funciona como um espaço reservado. O Spring acessa a lista de pessoas e substitui esse espaço pelo número real (o RG) daquela pessoa específica (ex: /pessoas/editar/5).
- **O Porquê**: Sem essa precisão, o Spring não saberia **quem** deve ser editado ou excluído no banco de dados. Ele utiliza esse número para filtrar a busca no MySQL.

3. A Necessidade do th:object="${pessoa}" **e o ID oculto**

O Spring exige a conexão entre th:action e th:object por um motivo de integridade do objeto:

- **O Contrato de Precisão**: Ao declarar th:object="${pessoa}", o Spring cria um "contrato". Ele avisa ao sistema que todos os campos com asterisco *{...} fazem parte daquela ficha específica.
- **O ID como Âncora**: Dentro desse objeto, o campo **th:field="*{id}"** (geralmente oculto) é fundamental. É ele quem diz ao Spring: "Não crie uma nova pessoa, apenas atualize esta pessoa que possui este ID".
- **O Fluxo**: O th:object prepara o molde completo (incluindo o ID invisível) e o th:action envia esse pacote fechado para o banco de dados.

Resumo do Comportamento do Spring (Funções Avançadas):

1. **Ele aumenta a precisão do sistema**: Ao amarrar o formulário a um objeto (`th:object`), o Spring impede que dados sejam salvos em lugares errados ou que usuários sejam duplicados por erro de referência.
2. **Ele simplifica o HTML**: Transforma um único `th:field` em vários atributos técnicos (id, name, value) necessários para o navegador entender o Java.
3. **Ele gerencia identidades**: Constrói URLs dinâmicas e utiliza o campo de ID dentro do objeto para diferenciar uma **Inclusão** (ID vazio) de uma **Alteração** (ID preenchido).
4. **Ele garante a integridade**: O Spring só envia o formulário se todos os campos estiverem de acordo com o "molde" definido na classe Java, garantindo que o banco de dados receba apenas informações válidas.

Configuração Inicial do Projeto